

nodegen et nodeview

Tutoriel : génération de nodeFiles pour L-HYPHEN,
algorithmes et améliorations possibles

Répertoire nodeGen/

4 octobre 2026

Résumé

nodegen produit le fichier de nœuds (*nodeFile*) lu par la commande **readNodeFile** de L-HYPHEN : un pavage de cellules polygonales d'un rectangle, éventuellement entaillé par une pré-fissure rectiligne. Il est conçu pour éviter les deux défauts rencontrés avec les maillages de **cellPrepro** : des barres extrêmement courtes, qui imposent un pas de temps minuscule en flexion, et une pré-fissure obtenue en retirant des cellules. À partir d'un fichier d'input existant pris comme modèle, il produit aussi un **input.txt** complet, directement exécutable par **run**. **nodeview** produit une vue SVG d'analyse de n'importe quel nodeFile. Ce document présente leur utilisation pas à pas, détaille les algorithmes et liste les améliorations envisageables.

Table des matières

1	Premiers pas	2
1.1	Compilation	2
1.2	Générer un premier échantillon	2
2	Paramètres	3
3	Visualiser et contrôler un nodeFile avec nodeview	6
4	Produire le fichier d'input l-hyphen	8
4.1	Input complet à partir d'un modèle (inputTemplate)	8
4.2	Lignes d'input seules	9
4.3	Mors : rangées de nœuds attrapées	9
4.4	Vérifications	10
5	Algorithmes	10
5.1	Germes	11
5.2	Régularisation de Lloyd	11
5.3	Pavage de Voronoï par découpe de demi-plans	11
5.4	Sommets partagés et types de sommets	12
5.5	Arêtes courtes : fusion ou étirement	12
5.6	Pré-fissure par symétrie	13
5.7	Pointe de fissure	13
5.8	Décalage des parois	14
6	Validation sur un essai d'ouverture	14

7 Limites et améliorations possibles	15
7.1 Anisotropie du réseau hexagonal	15
7.2 Contrôle du nombre de côtés	16
7.3 Géométrie	16
7.4 Robustesse et outillage	16

1 Premiers pas

1.1 Compilation

Les deux outils sont écrits en C++17 sans aucune dépendance :

```
cd nodeGen
make                # produit nodegen et nodeview
```

1.2 Générer un premier échantillon

Le fichier doc/tutorial_params.txt décrit un échantillon de même géométrie que le cas Frank-ouverture-fissure : 13,6 mm × 16 mm, environ 2200 cellules de 0,353 mm de diamètre équivalent, et une pré-fissure de 1,2 mm partant du milieu du bord gauche.

```
# Paramètres du tutoriel : échantillon du type Frank-ouverture-fissure (13.6 mm x 16 mm
, ~2200 cellules)

# Domaine
xmin 0.0032
ymin 0.002
Lx 0.0136
Ly 0.016

# Cellules
cellSize      3.53e-4  # diamètre équivalent visé
barWidth      2e-6     # écart entre parois voisines (= barWidth de readNodeFile)
lattice       hex      # réseau hexagonal perturbé
disorder      0.3      # perturbation des germes (fraction du pas du réseau)
lloydIterations 5      # régularisation de Lloyd
minEdgeRatio  0.4      # seuil des arêtes courtes (fraction de la longueur moyenne)
minSides      5        # fusion interdite si une cellule passait sous 5 côtés (é
tirement à la place)
seed          1

# Pré-fissure depuis le bord gauche, à mi-hauteur, longueur 1.2 mm
crack         0.0032 0.010 0.0044 0.010
tipInterface 1          # interface collée prolongeant la fissure au-delà de la
pointe (en cellules)
opening       2e-6      # écart supplémentaire entre les lèvres (> distGlue)
distGlue      2e-7      # valeur de distGcGlue dans l'input

# Lignes d'input l-hyphen (valeurs de Frank-ouverture-fissure)
cellProperties 2000.0 7.928638584420448e-4 79286385.84420447 0 # Kn Kr Mz_max
p_int
glueProperties 26373.626373626375 26373.626373626375 1.2575114788066131e-06
gripRows      1          # mors : une rangée de nœuds (celle du bord)
pullVelocity   3e-4      # vitesse imposée aux mors
nodeMass       8e-8      # masse des nœuds (pas de temps critique)

# Sorties
```

```

output work/nodefile.txt
svg    work/nodefile.svg
input  work/nodegen_input.txt

# Input complet : copie de modele_input.txt (input de Frank) avec les lignes du
# maillage remplacées
inputTemplate modele_input.txt work/input.txt

```

```

cd nodeGen/doc
../nodegen tutorial_params.txt

```

La génération prend moins d'un dixième de seconde. **nodegen** écrit le nodeFile (**output**), une vue SVG d'analyse (**svg**), les lignes d'input L-HYPHEN propres au maillage (**input**) et, ici, un fichier d'input complet construit à partir de **modele_input.txt** (**inputTemplate**, section 4) :

```

cd work && ../../run input.txt      # le calcul se lance directement

```

nodegen affiche un bilan :

```

nodegen
domaine      : [0.0032, 0.0168] x [0.002, 0.018]
cellules     : 2222 (visé 2223), noeuds : 13084
arêtes fusionnées : 34 (36 refusées : minSides), seuil 0.4 x 0.0002006
arêtes étirées  : 36 (fusion refusée pour garder >= 5 côtés)
barres        : l_moy = 0.0001986, l_min = 7.861e-05 (l_min/l_moy = 0.3957)
côtés         : 4: 1.8% 5: 9.1% 6: 87.8% 7: 1.4% 8: 0.0%
côtés (intérieur) : 4: 0.1% 5: 5.0% 6: 93.5% 7: 1.3% 8: 0.0% (2048 cellules)
parois        : 12902 collées, 182 libres (bords + fissure)
pré-fissure   : 8 parois ouvertes (barWidth + 2e-06), pointe effective en (0.004381, 0.01) au lieu de
(0.0044, 0.01)
fichiers      : work/nodefile.txt, work/nodefile.svg, work/nodegen_input.txt
input complet : work/input.txt (modèle modele_input.txt, fragments dans work/nodegen_input.txt)
mors bas      : 82 noeuds (1 rangée)
mors haut     : 79 noeuds (1 rangée)
               dt_crit(flexion) = 7.896e-07

```

Les lignes à surveiller :

- **barres** : le rapport $\ell_{\min}/\bar{\ell}$ doit rester de l'ordre de **minEdgeRatio**; c'est lui qui fixe le pas de temps critique de flexion dans L-HYPHEN ($\Delta t_{\text{crit}} \propto \ell_{\min}$);
- **côtés (intérieur)** : répartition des cellules par nombre de côtés, hors cellules de bord, qui sont coupées par le rectangle;
- **pré-fissure** : position effective de la pointe, qui est toujours un sommet du pavage;
- **lignes d'input** : nombre de noeuds dans chacun des mors haut et bas.

2 Paramètres

Le fichier de paramètres contient un mot-clé par ligne; tout ce qui suit **#** est un commentaire.

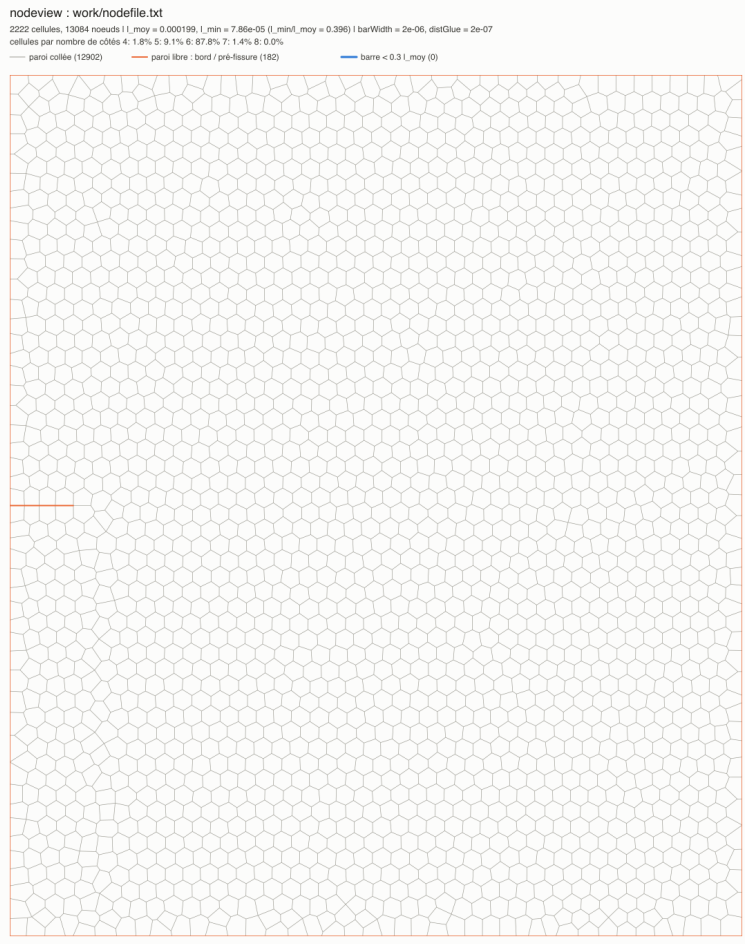


FIGURE 1 – Vue produite par `nodegen` pour `tutorial_params.txt`. En gris : parois collées ; en orange : parois libres (bords du domaine et pré-fissure) ; en bleu : barres courtes (aucune ici).

Mot-clé	Défaut	Rôle
<code>xmin, ymin</code>	0, 0	coin inférieur gauche du domaine
<code>Lx, Ly</code>	0.01	dimensions du domaine
<code>cellSize</code>	3.5e-4	diamètre équivalent visé; le nombre de cellules vaut $L_x L_y / (\pi d^2 / 4)$
<code>barWidth</code>	2e-6	largeur des barres, à reporter dans <code>readNodeFile</code>
<code>lattice</code>	<code>hex</code>	<code>hex</code> : réseau hexagonal perturbé; <code>random</code> : tirage aléatoire
<code>disorder</code>	0.3	perturbation des germes du réseau <code>hex</code> (fraction du pas)
<code>lloydIterations</code>	10	itérations de régularisation de Lloyd
<code>seed</code>	1	graine du générateur aléatoire
<code>minEdgeRatio</code>	0.3	seuil des arêtes courtes, en fraction de la longueur moyenne
<code>minSides</code>	5	une fusion d'arête ne peut faire passer une cellule sous <code>minSides</code> côtés; sinon l'arête est étirée
<code>crack x0 y0 x1 y1</code>	—	pré-fissure (segment), facultative
<code>opening</code>	2e-6	écart supplémentaire entre les lèvres de la fissure
<code>tipInterface</code>	1	longueur (en cellules) de l'interface collée qui prolonge la fissure
<code>distGlue</code>	2e-7	valeur de <code>distGcGlue</code> de l'input, pour les contrôles
<code>cellProperties</code>	—	<code>Kn Kr Mz_max p_int</code> reportés dans <code>readNodeFile</code>
<code>glueProperties</code>	—	<code>kn_coh kt_coh Gc</code> reportés dans <code>setGcGlueSameProperties</code>
<code>gripRows</code>	1	nombre de rangées de nœuds attrapées par chacun des mors bas et haut
<code>gripHeight</code>	—	hauteur explicite des mors (si <code>gripRows</code> n'est pas donné)
<code>pullVelocity</code>	3e-4	vitesse imposée aux mors (bas $-v$, haut $+v$)
<code>nodeMass</code>	—	masse des nœuds, pour estimer le pas de temps critique de flexion
<code>inputTemplate</code>	—	modèle [sortie] : input complet produit à partir d'un input existant (sortie par défaut <code>input.txt</code>)
<code>output, svg, input</code>		nodeFile, vue SVG, lignes d'input L-HYPHEN

Choisir la régularité. `lattice` et `disorder` règlent la forme des cellules (figure 2). Sur la géométrie du tutoriel, avec `lloydIterations` 5 et `minSides` 5, on obtient pour les cellules intérieures :

Réglage	4 côtés	5 côtés	6 côtés	7 côtés
<code>lattice random, lloydIterations 10</code>	0,3 %	44 %	47 %	8 %
<code>lattice hex, disorder 0.2</code>	0,1 %	4,5 %	95 %	0,4 %
<code>lattice hex, disorder 0.3</code>	0,1 %	5 %	94 %	1,3 %
<code>lattice hex, disorder 0.4</code>	0 %	16 %	83 %	0,8 %

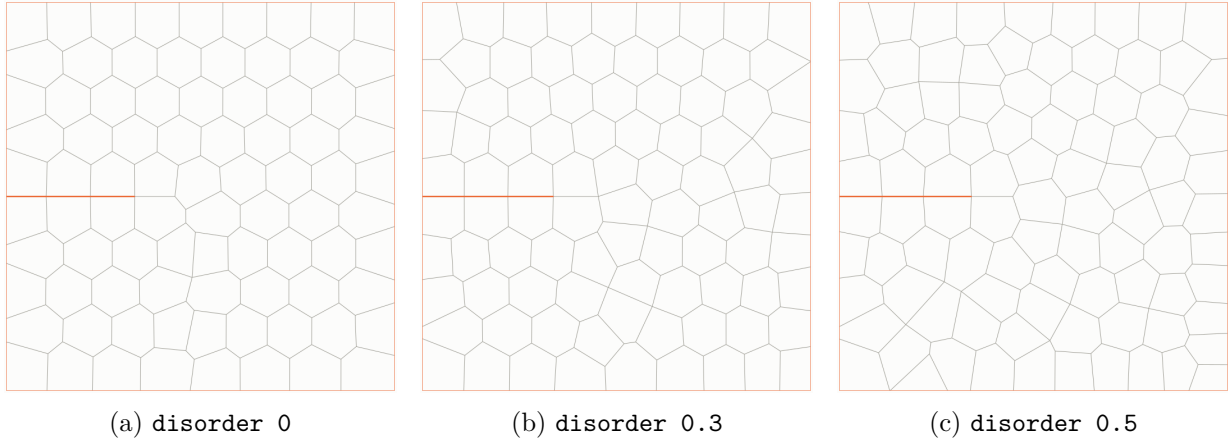


FIGURE 2 – Effet de **disorder** (réseau hexagonal, domaine de 3 mm, pré-fissure de 1 mm). Le long de la fissure, la rangée de cellules est symétrique ; plus loin, le réseau est régulier.

Pré-fissure. Le segment **crack** peut partir d'un bord (fissure débouchante) ou être entièrement intérieur, et avoir une orientation quelconque. **opening** doit être nettement supérieur à **distGlue** pour que les lèvres ne soient pas collées. **tipInterface** fixe la longueur de l'interface collée qui prolonge la fissure au-delà de la pointe (section 5.7).

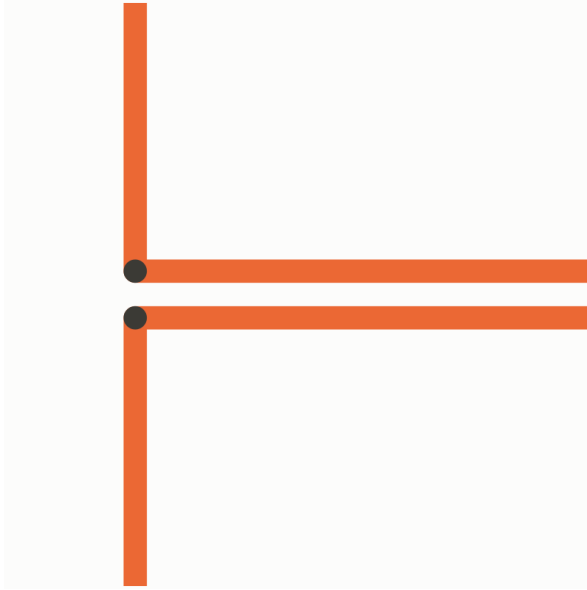
3 Visualiser et contrôler un nodeFile avec nodeview

```
nodeview nodefile.txt [-w barWidth] [-g distGlue] [-r ratio] [-box x0 x1 y0 y1]
                    [-nodes] [-bare] [-px largeur] [-o out.svg]
```

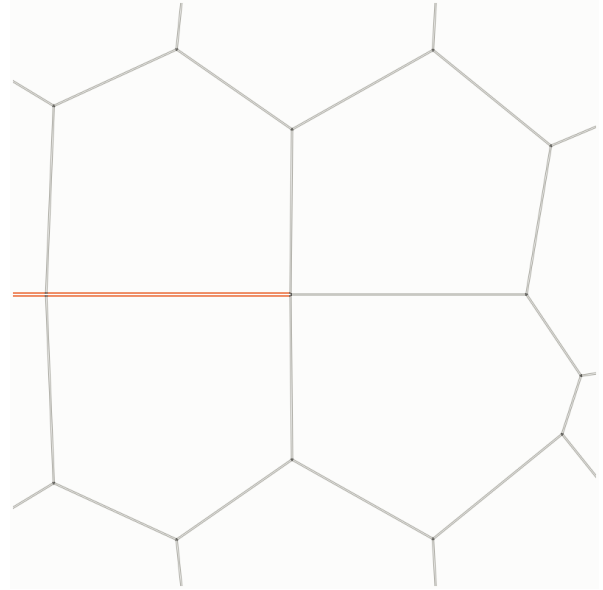
nodeview lit n'importe quel **nodeFile** (y compris ceux de **cellPrepro**) comme **readNodeFile** le fait, et produit un SVG (vectoriel : le zoom est illimité dans un navigateur). Chaque paroi est classée :

- **collée** si son milieu est à moins de **barWidth** + **distGlue** d'une barre d'une autre cellule
- c'est le critère de création des liens cohésifs de L-HYPHEN, et le nombre de parois collées est égal au nombre de « liens collés » affiché par **diagnostic.txt** ;
- **libre** sinon : bord du domaine, pré-fissure, trou ;
- les barres plus courtes que $\text{ratio} \times \bar{\ell}$ (défaut 0.3) sont surlignées en **bleu**.

Sans **-w**, la largeur des barres est estimée par la distance minimale entre nœuds de cellules différentes. **-box** restreint le dessin à une fenêtre (zoom), **-nodes** dessine les nœuds, **-bare** supprime titre et légende.



(a) Embouchure de la fissure (fenêtre de $50\text{ }\mu\text{m}$)



(b) Pointe de la fissure (fenêtre de $0,8\text{ mm}$)

FIGURE 3 – Zooms obtenus avec `-box` et `-nodes`. (a) Les lèvres de la fissure sont écartées de $\text{barWidth} + \text{opening} = 4\text{ }\mu\text{m}$, les parois collées de $\text{barWidth} = 2\text{ }\mu\text{m}$. (b) La fissure se termine sur un sommet et se prolonge par une interface collée.

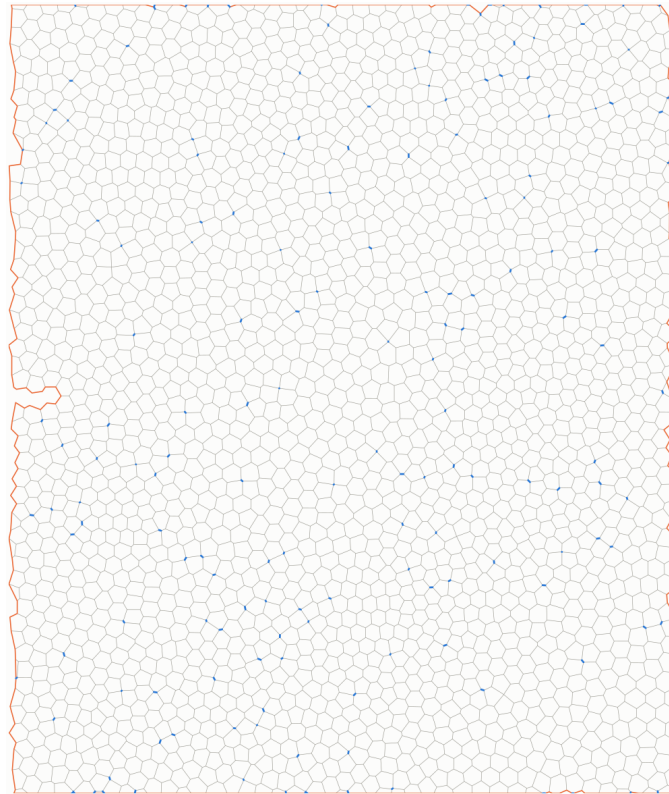


FIGURE 4 – `nodeview` appliqué au `nodeFile` de `Frank-ouverture-fissure` (produit par `cellPrepro`) : pré-fissure obtenue en retirant des cellules, bords latéraux irréguliers et 331 barres plus courtes que 0.3ℓ (en bleu), la plus courte mesurant 0.012ℓ .

4 Produire le fichier d'input l-hyphen

4.1 Input complet à partir d'un modèle (inputTemplate)

Le plus simple est de partir d'un fichier d'input existant, par exemple celui d'une étude précédente :

```
inputTemplate modele_input.txt work/input.txt
```

nodegen recopie le modèle dans **work/input.txt** en remplaçant les lignes qui dépendent du maillage, marquées # [nodegen]. Tous les autres réglages du modèle (pas de temps, raideurs de contact, masses, amortissements, sorties...) sont conservés, y compris les constantes **define** et les expressions.

Ligne du modèle	Traitement
readNodeFile	nouveau nodeFile (chemin relatif au répertoire de l'input produit) et barWidth ; Kn Kr Mz_max p_int de cellProperties , sinon recopiés du modèle
cleanShortBars	commentée (inutile)
2 setNodeControlInBox	boîtes recalculées sur le nouveau domaine, de hauteur fixée par gripRows (section 4.3); modes et valeurs repris du modèle, sauf si pullVelocity est donné
2 captureNodes	boîtes recalculées, noms de fichiers conservés
setGcGlueSameProperties	remplacée si glueProperties est donné, sinon recopiée
distGcGlue, glue	recopiée; nodegen vérifie qu'elle reste inférieure à opening
autres lignes	recopiées telles quelles

Les mors bas et haut sont reconnus comme les deux boîtes de plus petit et de plus grand $y_{\min}$. Si le modèle ne contient pas exactement deux **setNodeControlInBox** (ou deux **captureNodes**), ces lignes sont recopiées sans modification et un avertissement est affiché. **nodegen** affiche le nombre de nœuds de chaque mors et, si la masse des nœuds et k_r sont connus, le pas de temps critique de flexion; il refuse d'écraser le modèle.

Différences entre le modèle (input de **Frank-ouverture-fissure**) et l'input produit pour le tutoriel :

```
0a1,2
> # Input généré par nodegen à partir du modèle modele_input.txt et de tutorial_params.txt
> # Les lignes modifiées par nodegen sont marquées [nodegen] ; les autres sont celles du modèle.
27c29
< readNodeFile nodefile.txt 2e-06 2000.0 0.0007928638584420448 79286385.84420447 0
---
> readNodeFile nodefile.txt 2e-06 2000 0.000792863858442045 79286385.8442045 0 # [nodegen]
30c32
< cleanShortBars 0.3
---
> # cleanShortBars 0.3 # [nodegen] inutile : l_min/l_moy = 0.396
41,42c43,44
< setNodeControlInBox 0.0032085515974282354 0.01679607384100938 0.002 0.0024009500000000002
1 0.0 0 -3e-04
< setNodeControlInBox 0.0032085515974282354 0.01679607384100938 0.017599049999999998 0.018
1 0.0 0 3e-04
---
> setNodeControlInBox 0.0032 0.0168 0.002 0.00203869896891767 1 0.0 0 -0.0003 # [nodegen] mors bas
> setNodeControlInBox 0.0032 0.0168 0.0179594987793753 0.018 1 0.0 0 0.0003 # [nodegen] mors haut
45,46c47,48
< captureNodes bottom.txt 0.0032085515974282354 0.01679607384100938 0.002
0.0024009500000000002
< captureNodes top.txt 0.0032085515974282354 0.01679607384100938 0.017599049999999998
0.018
---
> captureNodes bottom.txt 0.0032 0.0168 0.002 0.00203869896891767 # [nodegen]
> captureNodes top.txt 0.0032 0.0168 0.0179594987793753 0.018 # [nodegen]
53c55
< setGcGlueSameProperties 26373.626373626375 26373.626373626375 1.2575114788066131e-06
```

```

---
> setGcGlueSameProperties 26373.6263736264 26373.6263736264 1.25751147880661e-06 # [nodegen]

```

L’input produit a été lancé sans autre modification que la durée simulée (20 000 pas) : aucune rupture au démarrage et 12902 liens collés, comme attendu.

4.2 Lignes d’input seules

Sans modèle, `nodegen` écrit toujours dans le fichier `input` (défaut `nodegen_input.txt`) les lignes d’input qui dépendent du maillage, à copier dans un fichier d’input aux endroits indiqués :

- `readNodeFile` avec le bon `barWidth` et les propriétés de `cellProperties` ;
- les mors : deux `setNodeControlInBox` sur toute la largeur (section 4.3), libres en x (mode 1, force nulle) et à vitesse imposée $\mp v$ en y (mode 0), avec le nombre de nœuds de chaque mors, et les deux `captureNodes` correspondants ;
- `distGcGlue` et `setGcGlueSameProperties` (propriétés de `glueProperties`) ;
- en commentaire, les statistiques du maillage, la position effective de la pointe et, si `nodeMass` est donné, le pas de temps critique de flexion $\ell_{\min} \sqrt{m/k_r}$.

Une ligne dont une valeur n’est pas connue est écrite en commentaire, avec des repères `<...>` à compléter. Pour `tutorial_params.txt` :

```

# ===== Lignes d’input générées par nodegen (tutorial_params.txt) =====
# Domaine [0.0032, 0.0168] x [0.002, 0.018], 2222 cellules, 13084 noeuds
# Barres : l_moy = 0.0001986, l_min = 7.861e-05 (l_min/l_moy = 0.3957): cleanShortBars est inutile
# Pré-fissure : de (0.0032, 0.01) à la pointe effective (0.004381, 0.01), lèvres écartées de barWidth +
# opening = 4e-06
# Les lignes commentées contiennent des valeurs <...> à compléter (ou à fournir dans le fichier de
# paramètres de nodegen : cellProperties, glueProperties).

# --- Échantillon (section SAMPLE)
#      fichier      barWidth  Kn  Kr  Mz_max  p_int
readNodeFile  work/nodefile.txt  2e-06  2000  0.000792863858442045  79286385.8442045  0
# setNodeMasses 8e-08 -> dt_crit(flexion) = l_min * sqrt(m/Kr) = 7.896e-07 s (dt <= 7.896e-08 conseillé)
# (masses, amortissements, cellContent... ici)

# --- Chargement : mors bas et haut, libres en x (mode 1 = force nulle), vitesse imposée en y (mode 0)
#      xmin  xmax  ymin  ymax  xmode  xvalue  ymode  yvalue
setNodeControlInBox  0.0032  0.0168  0.002  0.00203869896891767  1 0.0  0 -0.0003  # bas : 82 noeuds (1
rangée)
setNodeControlInBox  0.0032  0.0168  0.0179594987793753  0.018  1 0.0  0 0.0003  # haut : 79 noeuds (1
rangée)
captureNodes  bottom.txt  0.0032  0.0168  0.002  0.00203869896891767
captureNodes  top.txt    0.0032  0.0168  0.0179594987793753  0.018

# --- Collage (après les contrôles) : distGcGlue doit rester inférieur à opening = 2e-06
distGcGlue 2e-07
setGcGlueSameProperties 26373.6263736264 26373.6263736264 1.25751147880661e-06

```

L’ordre compte : `readNodeFile` ouvre la section de l’échantillon (avant les masses et les amortissements), les contrôles et `captureNodes` suivent, et le collage vient en dernier, une fois `distVerlet` défini. Le nombre de nœuds de chaque mors permet de vérifier que la hauteur des mors est adaptée ; `nodegen` signale un mors vide ou une pré-fissure qui entrerait dans un mors.

4.3 Mors : rangées de nœuds attrapées

Les mors bas et haut sont deux boîtes `setNodeControlInBox` de toute la largeur du domaine. Par défaut (`gripRows 1`), chacune attrape **une seule rangée de nœuds** : les nœuds du bord, tous situés à `barWidth/2` du bord du domaine. Les parois des cellules de bord restent alors libres de se déformer ; seuls leurs nœuds sur le bord sont pilotés.

Les rangées sont définies topologiquement : la rangée 1 est formée des sommets du bord, la rangée k des sommets situés à $k - 1$ barres du bord (les nœuds des cellules voisines au même sommet comptant pour un seul sommet). Une boîte ne sélectionne que par la hauteur ; `nodegen`

la fait s'arrêter à mi-distance entre le nœud le plus haut des `gripRows` premières rangées et le nœud suivant. Sur l'échantillon du tutoriel :

Mors bas	hauteur de la boîte	nœuds attrapés	dont nœuds des rangées suivantes
<code>gripRows 1</code> (défaut)	39 μm	82	0
<code>gripRows 2</code>	354 μm	253	45
<code>gripRows 3</code>	819 μm	651	303
<code>gripHeight 4e-4</code>	400 μm	312	—

La première rangée est attrapée exactement : le nœud suivant est à environ 78 μm du bord. Au-delà, un pavage de Voronoï n'a pas de rangées horizontales (le nœud le plus haut de la rangée 2 est plus haut que le nœud le plus bas de la rangée 3) : la boîte attrape toutes les rangées demandées, plus quelques nœuds bas des rangées suivantes, dont `nodegen` affiche le nombre. `gripHeight` impose directement une hauteur de boîte.

4.4 Vérifications

`cleanShortBars` est inutile avec un `nodeFile` produit par `nodegen`. Avec les mors d'une rangée, l'input du tutoriel a été lancé sur 20 000 pas : aucune rupture, et des forces égales et opposées sur les deux mors (−1,405 mN en haut, 1,403 mN en bas à $t = 1$ ms). Après un premier lancement, vérifier dans `diagnostic.txt` que le nombre de « liens collés » est égal au nombre de parois collées de `nodeview`, et que le rapport $\Delta t_{\text{crit}}/\Delta t$ de la ligne « flexion » est confortable. Sur l'échantillon du tutoriel, avec les paramètres mécaniques de `Frank-ouverture-fissure` :

	Frank (<code>cellPrepro</code>)	<code>nodegen</code> (tutoriel)
cellules / nœuds	2155 / 12798	2222 / 13084
$\ell_{\min}/\bar{\ell}$	0.012	0.40
$\Delta t_{\text{crit},\text{flexion}}$	$2,4 \times 10^{-8}$ s	$7,9 \times 10^{-7}$ s
liens collés (<code>diagnostic.txt</code>)	12542	12902
ruptures au démarrage (20 000 pas)	—	0

5 Algorithmes

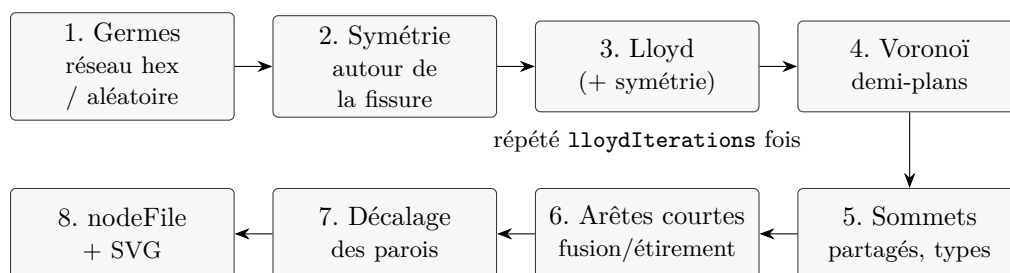


FIGURE 5 – Chaîne de traitement de `nodegen`.

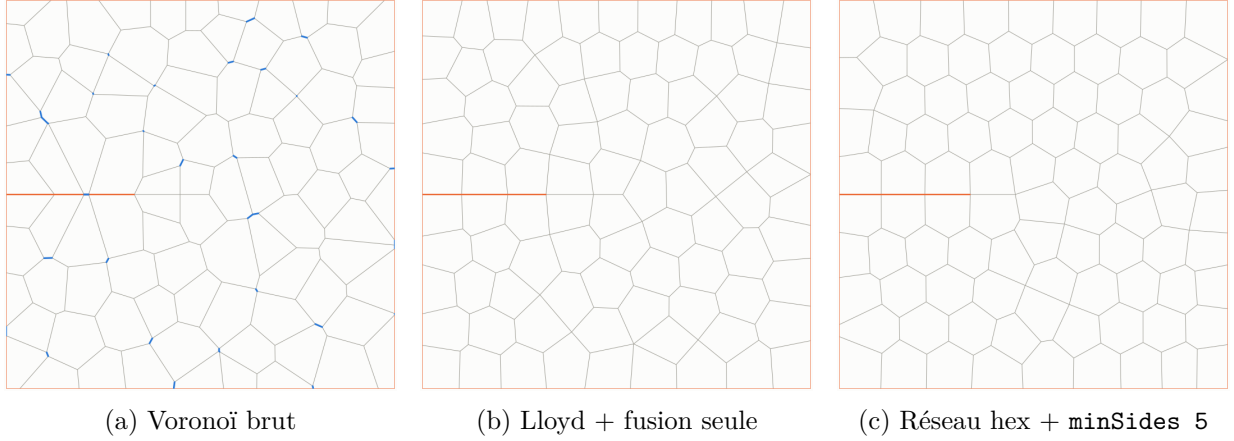


FIGURE 6 – Trois réglages sur un domaine de 3 mm. (a) Germes aléatoires, sans Lloyd ni traitement des arêtes courtes : nombreuses barres courtes (bleu), $\ell_{\min} = 0.01 \bar{\ell}$ et 29 % d’heptagones. (b) Lloyd (10 itérations) et fusion seule : $\ell_{\min} = 0.34 \bar{\ell}$, mais la fusion multiplie les pentagones (56 %). (c) Réseau hexagonal, **disorder** 0.3, fusion limitée par **minSides** 5 et étirement : $\ell_{\min} = 0.40 \bar{\ell}$, uniquement des pentagones et des hexagones.

5.1 Germes

Tirage aléatoire (lattice random). On vise $N = L_x L_y / A$ cellules d’aire $A = \pi d^2 / 4$ ($d = \text{cellSize}$), d’espacement moyen $s = \sqrt{A}$. Les germes sont tirés uniformément et acceptés s’ils sont à plus de $r_{\min} = 0.7s$ des germes déjà placés (recherche par grille) ; si la saturation est atteinte avant N , le complément est tiré sans contrainte.

Réseau hexagonal (lattice hex). Un réseau triangulaire de germes, de pas $a = \sqrt{2A/\sqrt{3}}$ (aire de la cellule hexagonale égale à A) et d’interrangée $a\sqrt{3}/2$, donne un pavage d’hexagones réguliers. Chaque germe est ensuite déplacé d’un vecteur aléatoire uniforme dans un disque de rayon **disorder** $\times a$. Le réseau est exprimé dans le repère de la fissure (**t**, **n**) (rangées parallèles à la fissure) et décalé pour qu’un sommet du réseau tombe sur la pointe. Le long de la fissure, les rangées sont placées symétriquement de part et d’autre de la ligne (section 5.6) ; ailleurs, elles alternent normalement.

5.2 Régularisation de Lloyd

À chaque itération, le pavage de Voronoï est calculé et chaque germe est déplacé au centroïde de sa cellule :

$$\mathbf{x}_i \leftarrow \frac{1}{|\Omega_i|} \int_{\Omega_i} \mathbf{x} \, dA.$$

Les itérations tendent vers un pavage de Voronoï centroïdal, aux cellules d’aires voisines et majoritairement hexagonales. La symétrie autour de la fissure est rétablie après chaque itération.

5.3 Pavage de Voronoï par découpe de demi-plans

La cellule du germe **p** est l’intersection du rectangle et des demi-plans $\{\mathbf{x} : (\mathbf{x} - \mathbf{m}) \cdot (\mathbf{q} - \mathbf{p}) \leq 0\}$, $\mathbf{m} = (\mathbf{p} + \mathbf{q})/2$, pour tous les autres germes **q**. Le polygone convexe est découpé successivement (algorithme de Sutherland–Hodgman) par les germes des anneaux de grille de rang $k = 0, 1, 2, \dots$ autour de **p** (pas de grille $h = s$). Après l’anneau k , les germes restants sont à plus de kh ; un germe à la distance D ne découpe la cellule que si $D < 2R$, où R est la distance de **p** au sommet le plus éloigné : on s’arrête dès que $kh \geq 2R$. Le coût est $O(N)$ pour une densité uniforme.

5.4 Sommets partagés et types de sommets

Chaque cellule est calculée indépendamment ; les sommets communs sont identifiés à $10^{-8}s$ près (table de hachage spatiale), ce qui donne une topologie cohérente : chaque arête est partagée par deux cellules (ou bordée par le domaine). Chaque sommet reçoit un type : bord gauche, droit, bas, haut, ligne de fissure, ou une combinaison (coin, embouchure de la fissure).

5.5 Arêtes courtes : fusion ou étirement

Les arêtes plus courtes que $\ell_c = \text{minEdgeRatio} \times \bar{\ell}$ sont traitées de la plus courte à la plus longue (figure 7).

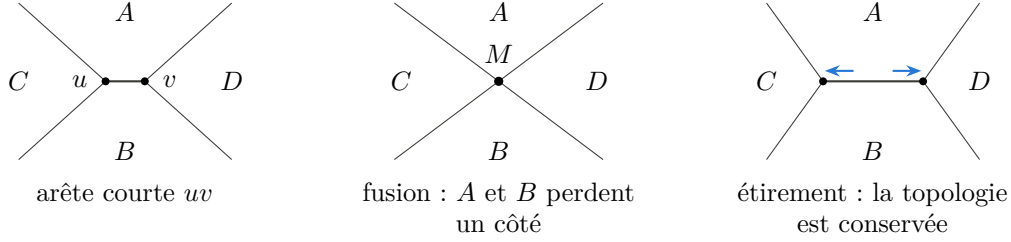


FIGURE 7 – Traitement d’une arête courte uv bordée par les cellules A et B .

Fusion. Les deux sommets sont réunis (union-find sur les sommets partagés, donc pour toutes les cellules à la fois). Les cellules A et B qui bordaient l’arête perdent un côté, C et D se touchent en un point. La fusion est refusée si une cellule concernée passait sous `minSides` côtés (et jamais sous 3). La position du sommet fusionné respecte les contraintes : un coin ou l’embouchure de la fissure reste fixe, un sommet de bord ou de fissure reste sur sa ligne (moyenne des sommets contraints de même type), sinon le sommet est placé au barycentre.

Étirement. Si la fusion est refusée, les deux sommets sont écartés le long de l’arête, $\mathbf{u} \leftarrow \mathbf{u} - \frac{\delta}{2}\mathbf{e}$, $\mathbf{v} \leftarrow \mathbf{v} + \frac{\delta}{2}\mathbf{e}$, avec $\mathbf{e}$ la direction de l’arête et $\delta = \ell_c - \ell$. Un sommet contraint ne se déplace que le long de sa ligne (projection), un coin ne bouge pas (l’autre sommet fait alors tout le chemin). Le déplacement est annulé s’il rendait une cellule voisine non convexe. Jusqu’à 20 passes sont effectuées ; les arêtes qui restent trop courtes sont signalées.

Pourquoi `minSides`. La fusion seule réduit systématiquement le nombre de côtés des deux cellules voisines de l’arête : sur un pavage aléatoire régularisé, elle produit 6 % de quadrilatères et 45 % de pentagones. Avec `minSides` 5, les fusions qui créeraient des quadrilatères sont remplacées par des étirements.

5.6 Pré-fissure par symétrie

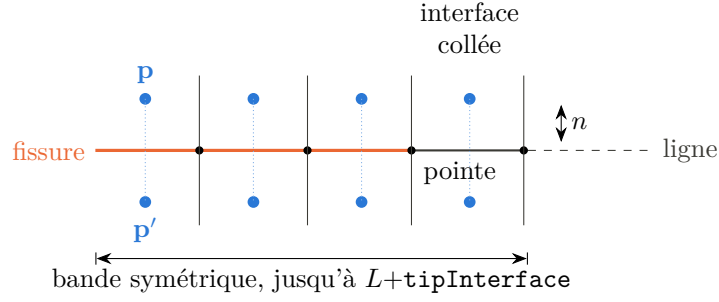


FIGURE 8 – Germes symétriques par rapport à la ligne de fissure : la médiatrice de $\mathbf{p}$ et de son symétrique $\mathbf{p}'$ est la ligne elle-même, qui est donc formée d'arêtes du pavage. La fissure est ouverte jusqu'à la pointe ; au-delà, la ligne se prolonge par une interface collée.

Dans une bande de demi-largeur $2s$ autour de la fissure, les germes du côté $n < 0$ sont supprimés et ceux du côté $n \geq 0$ sont dupliqués par symétrie. Un point $\mathbf{x}$ de la ligne est équidistant d'un germe et de son symétrique ; si les germes les plus proches de $\mathbf{x}$ sont tous dans la bande, l'ensemble des germes les plus proches est symétrique, et $\mathbf{x}$ est sur le bord de deux cellules symétriques. La ligne est donc exactement une suite d'arêtes du pavage : la fissure est rectiligne, aucune cellule n'est coupée ni retirée. Les germes trop proches de la ligne ($n < r_{\min}/2$) sont repoussés à $n = r_{\min}/2$ pour éviter des cellules très aplaties.

Ouverture. Le long de la fissure, les parois sont décalées de $w/2 + o/2$ au lieu de $w/2$ ($w = \text{barWidth}$, $o = \text{opening}$) : l'écart entre les lèvres vaut $w + o > w + \text{distGlue}$, donc aucun lien cohésif n'y est créé (figure 3a).

5.7 Pointe de fissure

La bande symétrique est prolongée au-delà de la pointe : la ligne se continue par une interface *collée* entre deux cellules, et la pointe est un sommet où se rejoignent quatre cellules, en face d'une interface plutôt que contre une cellule (figure 3b). La pointe est le sommet de la ligne le plus proche de (x_1, y_1) ; sa position effective est affichée.

Une ligne droite d'arêtes n'existe pas dans un pavage hexagonal : la rangée de pentagones qui borde la ligne doit se raccorder au réseau régulier, ce qui crée un défaut (quelques pentagones et heptagones) au bout de l'interface. `tipInterface` fixe la longueur de l'interface, donc la distance entre la pointe et ce défaut (figure 9). Au-delà, le réseau est régulier, afin de ne pas prolonger la fissure par une interface rectiligne qui serait un chemin de rupture privilégié.

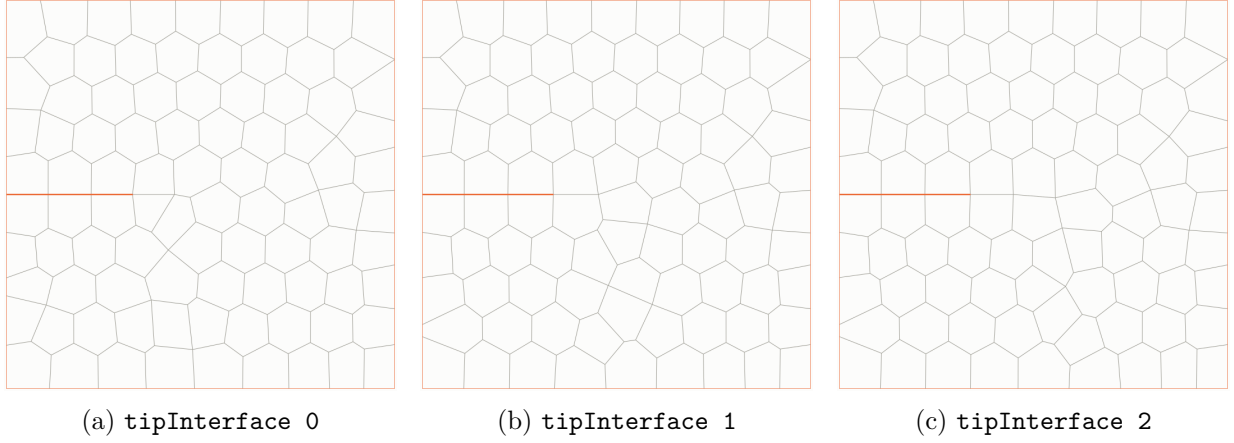


FIGURE 9 – Longueur de l’interface collée qui prolonge la fissure (en orange) au-delà de la pointe.

5.8 Décalage des parois

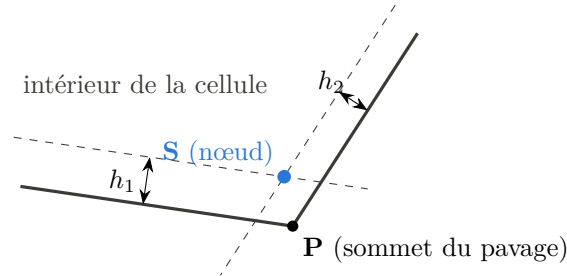


FIGURE 10 – Le nœud $\mathbf{S}$ est l’intersection des deux côtés adjacents décalés vers l’intérieur de h_1 et h_2 ($w/2$, ou $w/2 + o/2$ le long de la fissure).

Chaque cellule (polygone convexe orienté dans le sens direct) est rétrécie en décalant chacun de ses côtés vers l’intérieur : de $h = w/2$ en général, de $h = w/2 + o/2$ pour un côté de la fissure. Pour un sommet $\mathbf{P}$ entre deux côtés de normales intérieures unitaires $\mathbf{n}_1, \mathbf{n}_2$, le nœud $\mathbf{S}$ vérifie

$$\mathbf{n}_1 \cdot \mathbf{S} = \mathbf{n}_1 \cdot \mathbf{P} + h_1, \quad \mathbf{n}_2 \cdot \mathbf{S} = \mathbf{n}_2 \cdot \mathbf{P} + h_2,$$

système 2×2 résolu directement (si les deux côtés sont alignés, $\mathbf{S} = \mathbf{P} + \min(h_1, h_2) \mathbf{n}_1$). C’est la construction de `cellPrepro`. Deux cellules voisines décalent la *même* droite (le côté commun) de part et d’autre : leurs parois sont exactement à la distance w , condition de création des liens cohésifs par `glue / distGcGlue`. On vérifie enfin qu’aucun côté n’a été retourné par le décalage.

6 Validation sur un essai d’ouverture

Le `nodeFile` du tutoriel a été utilisé à la place de celui de `Frank-ouverture-fissure`, avec le même fichier d’input (sans `cleanShortBars`), ainsi qu’un `nodeFile` produit par une première version de `nodegen` (germes aléatoires, fusion seule, fissure butant sur une cellule).

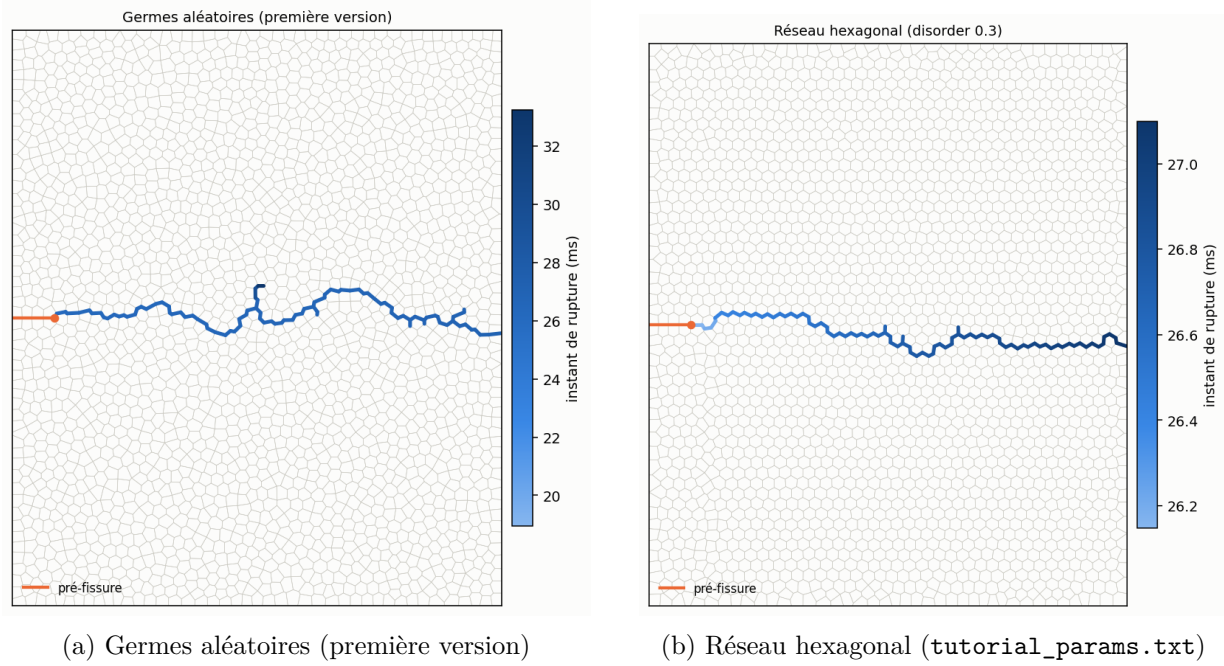


FIGURE 11 – Chemins de fissuration obtenus avec L-HYPHEN (liens rompus, colorés selon l’instant de rupture ; en orange, la pré-fissure).

	Frank (cellPrepro)	nodegen aléatoire	nodegen hexagonal
force au pic (mN)	−44,6	−44,2	−41,8
ouverture au pic (μm)	17,6	15,7	15,6
première rupture (ms)	29,33	18,94 (isolée), puis 26,21	26,15
fin de la propagation (ms)	36,59	33,25	27,10
ruptures	141	94	86
énergie rompue (μJ)	177	119	109
longueur rompue (mm)	25,9	17,4	16,3

La référence « Frank » est le calcul sur le nodeFile de **cellPrepro** au pas $\Delta t = 1,75 \times 10^{-8}$ s ; les deux maillages **nodegen** sont calculés à $\Delta t = 5 \times 10^{-8}$ s sans aucun nettoyage, avec l’amortissement de flexion $\alpha_b = 0.1$. Les maillages et les pré-fissures n’étant pas identiques, seule une comparaison qualitative a du sens :

- avec les deux maillages **nodegen**, la rupture s’amorce à la pointe de la pré-fissure ; avec le réseau hexagonal, les premiers liens rompus sont ceux de l’interface qui prolonge la fissure (à 0,16 mm puis 0,36 mm de la pointe) ;
- la fissure traverse l’échantillon en un seul chemin, sans bifurcation ; elle est sinueuse avec les germes aléatoires, mais suit en zigzag une rangée du réseau hexagonal et traverse l’échantillon en moins de 1 ms : c’est l’anisotropie discutée à la section suivante ;
- le chemin plus direct explique l’énergie et la longueur rompues plus faibles qu’avec le maillage de Frank, dont la fissure bifurque.

7 Limites et améliorations possibles

7.1 Anisotropie du réseau hexagonal

Avec **lattice hex**, les rangées du réseau sont parallèles à la fissure : la fissure se propage en zigzag le long d’une rangée (figure 11b), ce qui est un artefact de l’ordre du réseau. Pistes :

- augmenter `disorder` (0.4–0.5), au prix de davantage de pentagones et d’heptagones ;
- orienter le réseau hors de la zone de la fissure selon un angle différent (par exemple 30°, ou un angle aléatoire par « grain »), les raccords formant des joints de grains ;
- partir de germes aléatoires (isotropes) et contrôler le nombre de côtés par une optimisation topologique (paragraphe suivant) plutôt que par un réseau.

7.2 Contrôle du nombre de côtés

Le nombre de côtés n’est contrôlé qu’indirectement (réseau, `minSides`). Une optimisation explicite est possible : transitions T1 (rotation d’une arête courte, qui retire un côté à deux cellules et en ajoute un aux deux autres) choisies pour rapprocher chaque cellule de 5–6 côtés, ou pavage de puissance (Voronoi pondéré), dont les poids permettent d’agrandir les cellules à trop peu de côtés et de réduire les autres (loi de Lewis : l’aire d’une cellule croît avec son nombre de côtés).

7.3 Géométrie

- **Bords.** Les cellules de bord sont coupées par le rectangle (d’où les quadrilatères). Placer des germes symétriques par rapport aux bords donnerait des cellules de bord complètes et des bords rectilignes formés d’arêtes du pavage, comme pour la fissure.
- **Position exacte de la pointe.** La pointe est le sommet le plus proche du point demandé (à moins d’une demi-cellule) ; on pourrait ajuster le pas du réseau le long de la fissure pour qu’un sommet tombe exactement sur (x_1, y_1) .
- **Fissures multiples ou courbes, entailles en V :** symétrie par rapport à une ligne brisée ou à un arc, plusieurs bandes de symétrie (à condition qu’elles ne se chevauchent pas).
- **Domaines non rectangulaires, trous, inclusions :** découpe des cellules par un polygone quelconque au lieu du rectangle.
- **Distribution de tailles :** gradient de taille ou population bimodale, par une densité dans l’itération de Lloyd (centroïdes pondérés) ou par un pavage de puissance.

7.4 Robustesse et outillage

- L’étirement est local ; pour un pavage très désordonné (`disorder` ≥ 0.6), quelques arêtes peuvent rester trop courtes. Une relaxation globale des sommets (réseau de ressorts avec longueur minimale) serait plus robuste.
- Proposer d’autres chargements que la traction par mors (cisaillement, flexion trois points) ; reconnaître dans le modèle d’autres dispositions de boîtes de contrôle que deux mors haut et bas.
- Un visualiseur interactif (GLFW, comme `see2`) pour naviguer dans les grands maillages.
- Des tests automatiques : topologie (chaque arête intérieure partagée par exactement deux cellules), écartement des parois, rectitude de la fissure, reproductibilité à graine fixée.

Régénérer les figures

Toutes les figures de maillage, ainsi que le bilan affiché et les lignes d’input reproduits dans ce document, sont produits par `doc/make_figures.sh` (nécessite `inkscape` et `ImageMagick`) à partir de `doc/tutorial_params.txt` ; les fichiers intermédiaires sont dans `doc/work/`. Les chemins de fissuration proviennent de calculs L-HYPHEN et les valeurs de la section de validation de `doc/validation_results.tex`.